The Journal of Open Source Software

ModularML: A Backend-Agnostic, Declarative Framework for Reproducible and Modular Machine Learning Experiments

Ben Nowacki [1], **Tingkai Li** [1], **Sina Navidi** [1], **Mohammad Mundiwala**[1], **Hui Hua** [1], **Fei Miao**[1], **and Chao Hu** [1¶]

1 School of Mechanical, Aerospace, and Manufacturing Engineering, University of Connecticut, Storrs, CT 06269, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Modern machine learning (ML), particularly deep learning, has become central to scientific research across domains such as energy systems, materials science, structural health monitoring, and biomedical engineering. However, core ML frameworks primarily focus on model implementation and optimization, leaving experiment structure, data partitioning, and reproducibility patterns to user-defined scripts. As a result, domain researchers often rely on fragmented scripts, backend-specific conventions, and ad hoc experiment management workflows that limit reproducibility and collaboration.

`ModularML` is an open-source Python framework designed to address this gap. It provides a backend-agnostic, declarative, and fully serializable architecture for ML experimentation, enabling users to define complete workflows—including data curation, feature processing, model architecture, training strategy, loss configuration, and evaluation—using modular, composable components that serialize into a single reproducible artifact.

At its core, `ModularML` introduces an abstraction for structured, backend-agnostic data handling and splitting; a directed acyclic graph (DAG) of model nodes with support for shared or node-specific optimization; multi-phase training orchestration with configurable freeze/unfreeze policies and phase- and node-specific loss definitions; and full pipeline serialization and visualization for transparent sharing and inspection of experiments. By separating experiment definition from backend implementation, `ModularML` allows researchers to collaborate and audit ML studies without requiring expertise in `PyTorch` (Ansel et al., 2024), `Keras` (Chollet & others, 2015), or other specific ML libraries.

Statement of need

ML workflows in academia differ substantially from typical industry-focused ML development. Scientific researchers emphasize experimentation with domain-specific feature selection, sampling, and model architecture. Existing libraries such as `PyTorch Lightning` (W. Falcon & The PyTorch Lightning team, 2019), `Catalyst` (Kolesnikov, 2018), `FastAI` (Howard & others, 2018), and `PyTorch Ignite` (Fomin et al., 2020) can streamline training loops, but typically remain tied to a single backend, prioritize development productivity over experimentation standardization, and depend on implicit code-based configurations with limited traceability of dataset splitting.

In contrast, `ModularML` is built around the principle of *configuration-as-contract*: the entire ML experiment is represented as a structured, inspectable object. This design offers three

major benefits:

1. **Reproducibility through serialization.** Entire experiments, from initial data curation to multi-stage training and evaluation, can be saved and shared as a single file. A collaborator can load this artifact and inspect the full pipeline without having to trace the source code.
2. **Backend abstraction.** Model nodes are agnostic to the backend in which the underlying model is defined. `ModularML` does not replace existing ML libraries such as `PyTorch`, `Keras`, or `scikit-learn` (Pedregosa et al., 2011); rather, it complements them by providing a unified experiment structure around backend-specific model implementations. This backend-agnostic design supports reuse of collaborator models regardless of which library was used to implement them.
3. **Transparency for non-ML experts.** Built-in `summary` and `visualize` methods for feature sets, model graphs, and experiment execution phases allow users and code reviewers to quickly inspect data splits, model architecture, loss routing, and training sequencing without needing to understand backend-specific implementation details.

These features directly address common reproducibility and collaboration challenges in research, while still supporting a comprehensive suite of modeling techniques and existing ML libraries. In particular, `ModularML` is intended to serve as an experiment-structure layer rather than a replacement for backend frameworks: users can continue relying on established libraries for model implementation and numerical execution while using `ModularML` to define, serialize, and inspect the full experimental workflow. This makes the framework especially useful in collaborative research settings where model design, data partitioning, and training logic must be reviewed across users with different levels of ML software expertise.

State of the field

General-purpose deep learning frameworks (e.g., `PyTorch`, `Keras`) streamline model training but typically assume homogeneous backends and single-phase workflows. Data-centric libraries (e.g., `Apache Arrow` ("Arrow," n.d.), `Hugging Face` (Lhoest et al., 2021)) manage datasets but leave sampling and experiment scheduling to custom user code. Experiment managers (e.g., `MLflow` (Zaharia et al., 2018), `Weights & Biases` ("Wandb," n.d.)) track runs but do not define how data flows through modular graphs. `ModularML` sits between these layers. It provides concrete abstractions for data (`modularml.FeatureSet`), sampling logic (`modularml.Sampler`), modeling (`modularml.ModelGraph` composed of `modularml.ModelNodes`), losses (`modularml.AppliedLoss`), and orchestration (`modularml.ExperimentPh`) while remaining backend-agnostic and lightweight enough for research scripts or notebooks.

Software design

The `ModularML` architecture provides abstractions for data storage and processing, graph-based modeling, and execution of distinct training sequences (Figure 1). Rather than embedding these responsibilities within custom training scripts, `ModularML` separates them into explicit, composable components that can be independently configured, inspected, and serialized. This separation enables rapid experimentation with data pipelines, model architectures, and training strategies while preserving reproducibility and traceability across the entire ML workflow.

The Journal of Open Source Software

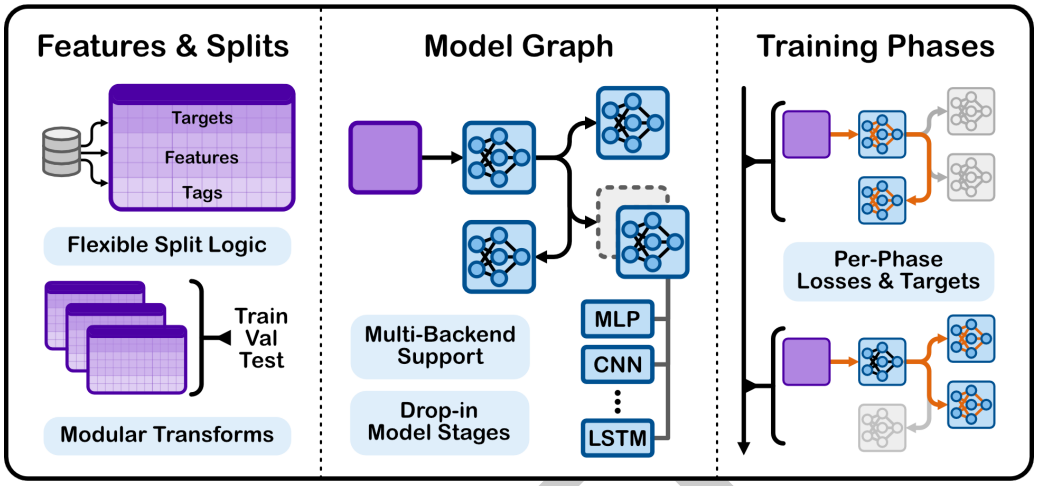

Figure 1: ModularML architecture overview: FeatureSet curation into samples with feature, target, and tag domains; flexible ModelGraph construction for rapid experimentation of model topology; and multi-phase training workflows sequenced within a single Experiment container.

FeatureSets

A `modularml.FeatureSet` organizes data into three intent-driven domains—features, targets, and tags—reflecting the core design philosophy of the data abstraction. Features represent model inputs (what the model learns from), targets represent model outputs (what the model is trained to predict or reproduce), and tags store optional metadata associated with each sample. Each sample, defined by its feature, target, and tag attributes, is assigned a globally unique identifier to ensure explicit traceability throughout the entire life cycle of an experiment. This identifier propagates through splitting, sampling, batching, model execution, and evaluation, enabling transparent lineage tracking and reproducible analysis.

Data within a `modularml.FeatureSet` is stored in backend-agnostic containers supporting `NumPy` (Harris et al., 2020) arrays, `Pandas` (team, 2020) dataframes, `PyTorch` tensors, and `TensorFlow` (Abadi et al., 2015) tensors. All downstream operations, such as splitting, subsetting, and batching, operate using no-copy views of the underlying data. This design makes split definitions explicit and inspectable, reduces memory overhead, and minimizes the risk of data leakage or unintended experimentation bias.

Sampling

All subclasses of `modularml.Sampler` consume a `modularml.FeatureSet`, or subset views of one, and emit aligned batches, supporting stratification, grouping, and multi-role sampling needed for contrastive or paired training schemes. By separating sampling logic from model execution, `ModularML` makes data selection strategies explicit and reproducible rather than embedding them within custom training loops. This design allows researchers to experiment with different batching and sampling approaches without modifying the model or training code.

ModelGraph

The full experiment model is represented as a DAG of interconnected `modularml.ModelNodes`. Each node wraps a user-defined or built-in ML model and exposes standardized interfaces for construction, optimization, and freezing. Specialized node types, such as `modularml.MergeNode`, extend this functionality by supporting multiple inputs and configurable merge operations, including concatenation, aggregation, and padding. Together, these nodes form a `modularml.ModelGraph`, which manages dependency resolution and executes forward and backward passes in topological order. This graph-based design

separates model topology from training logic, enabling rapid experimentation with branching architectures, multi-stage pipelines, and hybrid systems that combine learned and classical ML components.

Losses and phases

`modularml.AppliedLoss` binds objectives to specific nodes within a `modularml.ModelGraph`, enabling composite-loss training and targeted loss aggregation across different portions of a model. These losses are tracked throughout execution, providing detailed records for post-training analysis and comparison of experimental configurations.

Model execution is organized into a sequence of `modularml.ExperimentPhase` objects. Each phase defines a reproducible unit of execution, including data sampling, loss definitions, callbacks, and training or evaluation behavior. Built-in phase types include iterative training (`modularml.TrainPhase`), single-pass fitting workflows (`modularml.FitPhase`), and inference-only evaluation (`modularml.EvalPhase`), providing a consistent interface for constructing complex multi-phase ML workflows.

Experiment orchestration

The `modularml.Experiment` class serves as the top-level container for an ML workflow, binding together one or more `modularml.FeatureSets`, a `modularml.ModelGraph`, and a sequence of execution phases. By explicitly separating experiment definition from execution, `ModularML` supports workflows such as pretraining, fine-tuning, and evaluation while retaining all information required to reproduce each stage. The `modularml.Experiment` also provides serialization, checkpointing, and execution tracking, allowing complete experiments to be exported, shared, and reloaded as self-contained artifacts.

Research impact statement

`ModularML` provides structured abstractions for scientists prototyping hybrid ML systems that combine learned encoders, classical regressors/classifiers, and domain-specific samplers. By guaranteeing that every component can be serialized, checkpointed, and replayed, the framework supports reproducible experiments and facilitates sharing of trained graphs or datasets between collaborators. Its backend-agnostic graph execution simplifies comparisons across `PyTorch`, `TensorFlow`, and `scikit-learn` implementations, encouraging rigorous benchmarking and cross-validation in applied research domains.

The unique contributions can be summarized as follows:

1. **Full pipeline serialization.** Rather than serializing only model weights, `ModularML` serializes data definitions with assigned unique identifiers, split logic, sampling configuration, graph topology, per-phase loss routing, and training and evaluation configurations. This enables complete auditability of published ML studies, as shown in Figure 2.
2. **Backend-agnostic DAG modeling.** By supporting backend-agnostic modeling, `ModularML` extends adoption of existing ML libraries rather than replacing them, reducing friction between works that use different packages.
3. **Declarative syntax.** All aspects of an ML experiment are constructed via declarative, configuration-driven objects. This makes experiment definitions explicit, simplifies review, and supports systematic iteration across training techniques and model architectures.
4. **Built-in visualization for validation.** Visual summaries help detect data leakage, misconfigured sampling, incorrect loss routing, unexpected model topologies, and training sequencing issues (Figure 2). This enables validation at definition time, ensures execution matches intent, and lowers the barrier for domain scientists to review ML-based results.

The Journal of Open Source Software

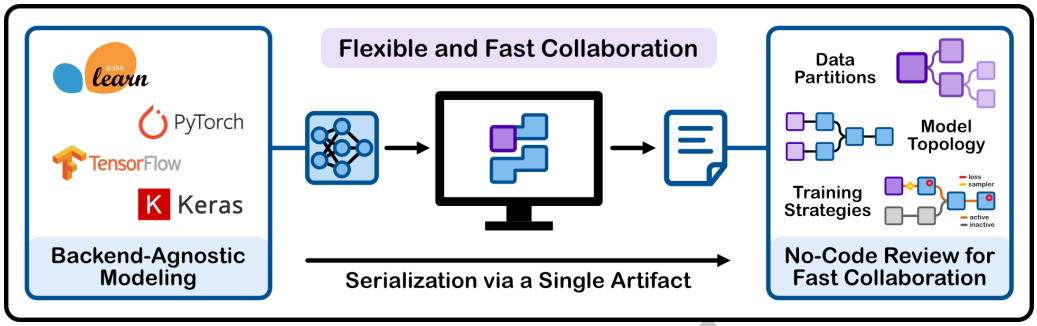

Figure 2: Backend-agnostic and fully serializable experiment workflow in ModularML with built-in visualization utilities.

Mathematics

`ModularML` does not introduce new mathematical formulations; instead, it codifies well-established training loops, loss aggregation, and optimizer steps across common ML backends. Any model- or loss-specific mathematics is delegated to user-defined modules or external libraries, ensuring that `ModularML` remains an orchestration layer that provides continued support for existing ML packages.

AI usage disclosure

Generative AI tools were used in a limited capacity during the development of this software, confined to helping structure docstrings and scaffold unit tests. All AI-assisted contributions were reviewed and verified by the authors. AI was not used in the writing of this manuscript.

Acknowledgements

We thank the `ModularML` contributor community for feature ideas, bug reports, and documentation improvements, and acknowledge Professor Ryan Cooper for guidance in navigating the open-source environment.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://www.tensorflow.org/

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., … Chintala, S. (2024, April). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. https://doi.org/10.1145/3620665.3640366

Arrow. (n.d.). In *GitHub repository*. GitHub. https://github.com/apache/arrow

Chollet, F., & others. (2015). *Keras*. https://keras.io

Fomin, V., Anmol, J., Desroziers, S., Kriss, J., & Tejani, A. (2020). High-level library to help with training neural networks in PyTorch. In *GitHub repository*. GitHub. https://github.com/pytorch/ignite

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Howard, J., & others. (2018). *Fastai*. GitHub. https://github.com/fastai/fastai

Kolesnikov, S. (2018). Catalyst - accelerated deep learning r&d. In *GitHub repository*. GitHub. https://github.com/catalyst-team/catalyst

Lhoest, Q., Villanova del Moral, A., Platen, P. von, Wolf, T., Šaško, M., Jernite, Y., Thakur, A., Tunstall, L., Patil, S., Drame, M., Chaumond, J., Plu, J., Davison, J., Brandeis, S., Sanh, V., Le Scao, T., Canwen Xu, K., Patry, N., Liu, S., … Delangue, C. (2021). Datasets: A Community Library for Natural Language Processing. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 175–184. https://aclanthology.org/2021.emnlp-demo.21

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

team, T. pandas development. (2020). *Pandas-dev/pandas: pandas* (latest). Zenodo. https://doi.org/10.5281/zenodo.3509134

W. Falcon, & The PyTorch Lightning team. (2019). PyTorch lightning (version 1.4). In *GitHub Repository*. GitHub. https://doi.org/10.5281/zenodo.3828935

Wandb. (n.d.). In *GitHub repository*. GitHub. https://github.com/wandb/wandb

Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, *41*, 39–45. https://api.semanticscholar.org/CorpusID:83459546